Design Rationale REQ 1: Crop Watering, Harvesting and Consumption

Crop Class Hierarchy

Assignment 1 already provided parsnip and tomato crops. REQ1 adds strawberry, cauliflower and melon crops, and extends parsnip with watering behaviour. These changes require shared interaction rules while preserving the differences between crop species.

Design 1:

Keep each crop as an independent Ground subclass. Each class implements its own interaction checks, stamina payment and removal behaviour alongside its species-specific rules.

Pros	Cons
Localised behaviour: Each crop can be read independently, with fewer abstraction layers to follow.	Repeated logic: Reach checks, stamina checks and removal-to-dirt behaviour would be copied across classes. A common rule change would require several edits and could produce inconsistent behaviour.

Design 2:

Introduce Crop for shared harvesting checks, stamina payment and removal. WaterableCrop extends Crop and implements Waterable, adding carried-tool checks, watering costs and a separate watering count for each plant. Concrete crops supply harvestAt() and, where applicable, waterAt().

Pros	Cons
DRY and OCP: Shared workflows are implemented once. A new crop can supply its own hooks without rewriting payment or removal logic. Tomato can remain harvestable without acquiring watering behaviour.	Additional indirection: Understanding one interaction requires following both the concrete crop and its base classes. Changes to a shared workflow can affect several species and therefore need regression checks.

Selected design: Design 2. The shared entry methods validate the action before spending stamina, then call the species-specific hook. Crop owns harvesting and removal; WaterableCrop owns watering. Neither base class represents a growth cycle: the crops are already grown.

Consumption and Timed Health Effects

Design 1:

Keep immediate food effects and multi-turn healing or poisoning in the crop or food class. That class would also have to arrange future updates after the food has been consumed.

Pros	Cons
Fewer separate classes: The rule that produces an effect and the code describing that effect remain close together.	Mixed responsibilities: Food production, consumption and effect timing become coupled. Removing the consumed item makes it harder to keep the delayed effect alive, and repeated timer logic would be needed for similar foods.

Design 2:

Use ConsumableProduce to check inventory ownership and remove the original item before calling the concrete consume() method. Strawberry attaches a separate TimedHealthEffect implementing the engine Status interface. The engine updates the effect and removes it when its duration ends.

Pros	Cons
SRP and reuse: Produce classes define what eating does; TimedHealthEffect owns the per-turn health change and remaining duration. Fresh and rotten strawberries share the same effect class with different constructor values.	Lifecycle dependency: The result depends on the engine status-update schedule. Testing must cover future turns and expiration, not just the immediate consume call. More objects and classes are involved.

Selected design: Design 2. Fresh strawberries attach a +1 HP effect for two future turns, while rotten strawberries attach a -1 HP effect for three. Immediate effects, such as cauliflower healing, remain in the concrete produce class rather than being forced into a timer.

This separation also allows the effect to continue after the original strawberry has left the inventory. Each consumed strawberry creates its own status instance, so one item does not overwrite another item’s remaining duration.

Action Dependencies

Design 1:

Make watering, harvesting and consumption actions recognise individual concrete crop or food classes and choose the corresponding behaviour through type-specific branches.

Pros	Cons
Explicit dispatch: For a small fixed set of products, each supported type and its behaviour can be seen in one action implementation.	Concrete coupling: Every added crop or food type can require another action branch. The action duplicates knowledge already held by the target and becomes harder to extend.

Design 2:

Keep WaterAction, HarvestAction and ConsumeAction dependent on Waterable, Harvestable and Consumable respectively. Each action delegates to its target through the relevant interface; the target provides the actual behaviour.

Pros	Cons
DIP and OCP: Actions use capability contracts rather than a list of concrete species. A new implementation of a supported capability can reuse the action without adding a type-specific branch.	Contract discipline: Implementations must interpret each capability consistently. The current Waterable and Harvestable interfaces share requirementMet(), so different watering and harvesting conditions must still be checked in their execution paths.

Final Design Summary

Design 2 is used in each area. The crop hierarchy centralises shared interaction rules while concrete crops retain their own outcomes. ConsumableProduce handles ownership and consumption, and TimedHealthEffect handles delayed health changes. The action classes delegate through capability interfaces.

Compared with Assignment 1, the main change is the separation of shared workflows from species-specific behaviour. This reduces duplicated checks and supports new crops without expanding action classes. The trade-off is a larger set of collaborating classes whose contracts and turn timing must remain consistent.